Image Compression Analysis using DCT and DFT Transforms

Mohamed Mahmoud 202100071
Alya Marwan 202200375

1 Principles of DCT and DFT

1.1 DFT Principle

For any processing to be done on a computer, it needs to be discrete in both time and frequency. That was the base for the DFT, starting by discretizing the Continuous Time Fourier Transform to produce the Discrete Time Fourier Transform, which is periodic in frequency due to the discretization in time. The DTFT is then sampled at N equally spaced frequency points, producing the Discrete Fourier Transform. Which is basically a sample-by-sample correlation between x(n) and a group of sines and cosines, checking for a specific frequency in x(m). The frequencies seen in the DFT are multiples of complete cycles across the N samples. The output of the DFT is generally complex because it represents both magnitude and phase. For real-valued signals such as grayscale images, in this case, the DFT exhibits complex conjugate symmetry. Where only a number of frequency coefficients is unique, if N is even, only N/2+1 coefficients contain unique information, while the rest can be constructed using conjugate symmetry. If N is odd, only (N+1)/2 coefficients need to be calculated. However, the DFT always assumes that the input signal is periodic, which in general is not the case; real images aren't periodic, and their boundaries don't match. So when the DFT forces the edges to connect, it creates artificial discontinuities at the edges, introducing high-frequency components that do not exist in the original signal.

1.2 DCT Principle

Given a signal $x[n]$, we could previously use DFT to transform it to frequency domain where $x[f] = \sum x[m] \cdot e^{-j2\pi kn/N}$ over $k = 0$ to $N-1$ and to get back to time representation $\rightarrow$ IDFT is then $x[n] = \sum \frac{X[k]}{N} \cdot e^{j2\pi kn/N}$ over $k = 0$ to $N-1$. Obviously, this yields complex numbers/sinusoids which are not ideal, nor cost-effective, for image processing. It won't matter a lot when working on LTI systems, but for compression tasks, we want to avoid complex numbers as much as possible. Loss of information happens when we quantize the coefficients, and working with complex numbers makes this loss worse. On the other hand, with DCT, the signal is represented as a sum of cosine functions oscillating at different frequencies, so losing a part of the signal is less effective, as the other half represents the same information. DCT is a similar transform that yields real numbers and cosines instead of complex exponentials. In DCT, a large amount of signal information is concentrated mostly in lower frequencies, which makes it more suitable for compression tasks, as we can lose parts of the higher frequency components with less quality loss. (As

we learned in information theory, higher frequencies represent less important information, and the more probable of occurrence, the less information they carry).

2 Experimental Setup

2.1 Block Size

In this project, the input image was first converted to grayscale and resized to 256×256 pixels. The image was then divided into non-overlapping square blocks of the following sizes:

- 4×4
- 8×8
- 16×16

2.2 Retention Ratio

After applying the transforms, only a percentage of the largest-magnitude frequency coefficients were retained for reconstruction. The remaining coefficients were set to zero.

DFT: 10%, 25%, 50%

DCT: 10%, 30%, 50%, 70%, 90%

2.3 Metrics Used

To evaluate the performance of both transforms, a numerical metric was used, which is the Mean Squared Error, the lower the better indicating better reconstruction quality. Aside from that, visual metrics were also used, such as tables and comparison grids.

2.4 Code

```
#divide into 16x16 blocks
blksize =16
blocks = []
for i in range(0, image_resized.shape[0], blksize):
    for j in range(0, image_resized.shape[1], blksize):
        block = image_resized[i:i+blksize, j:j+blksize]
        blocks.append(block)
blocks = np.array(blocks)
plt.imshow(blocks[0], cmap='gray')
plt.title('first block')
plt.axis('off')
plt.show()
#then repeat the same steps as before with 4x4 blocks
```

```
dct_rows = np.zeros_like(blocks)
dct_blocks = np.zeros_like(blocks)
for r in range (blocks.shape[0]):
    dct_rows[r] = dct(blocks[r], axis=1, norm='ortho')
for c in range (blocks.shape[0]):
    dct_blocks[c]=dct(dct_rows[c], axis=0, norm='ortho')
dct_blocks
#flatten first the blocks
flattened_dct_blocks = dct_blocks.reshape(dct_blocks.shape[0], -1
)
flattened_dct_blocks
#magnitudes calculation
magnitudes = np.abs(flattened_dct_blocks)
magnitudes
#now we try different compression ratios by zeroing small coefficients
def compress_dct_blocks(dct_blocks, keep_fraction):
    flattened = dct_blocks.reshape(dct_blocks.shape[0], -1)
    magnitudes = np.abs(flattened)
    compressed_blocks = np.zeros_like(flattened)
    for i in range(magnitudes.shape[0]):
        thresh = np.percentile(magnitudes[i], (1 - keep_fraction) * 100)
        compressed_blocks[i] = np.where(magnitudes[i] >= thresh, flattened[i], 0)
    return compressed_blocks.reshape(dct_blocks.shape)
compression_ratios = [0.1, 0.3, 0.5,
                      0.7, 0.9]
compressed_dct_blocks_list = []
for ratio in compression_ratios:
    compressed_blocks = compress_dct_blocks(dct_blocks, ratio)
    compressed_dct_blocks_list.append(compressed_blocks)
compressed_dct_blocks_list
#reconstruct images from compressed dct blocks
def reconstruct_image_from_dct_blocks(dct_blocks, image_shape, blksize):
    reconstructed_image = np.zeros(image_shape)
    idx = 0
    for i in range(0, image_shape[0], blksize):
        for j in range(0, image_shape[1], blksize):
            block_dct = dct_blocks[idx]
            block_idct_rows = idct(block_dct, axis=0, norm='ortho')
            block_idct = idct(block_idct_rows, axis=1, norm='ortho')
            reconstructed_image[i:i+blksize, j:j+blksize] = block_idct
            idx += 1
    return reconstructed_image
reconstructed_images = []
for compressed_blocks in compressed_dct_blocks_list:
    reconstructed_image = reconstruct_image_from_dct_blocks(compressed_blocks,
↪   image_resized.shape, blksize)
    reconstructed_images.append(reconstructed_image)
#display reconstructed images
for i, ratio in enumerate(compression_ratios):
```

```
    plt.imshow(reconstructed_images[i], cmap='gray')
    plt.title(f'Reconstructed Image with {int(ratio*100)}% DCT Coefficients (16x16)')
    plt.axis('off')
    plt.show()
#now i'll calc mean squared error for each reconstructed image
#mse for 4x4 blocks
mse_values = []
norm_image = np.linalg.norm(image_resized)
for reconstructed_image in reconstructed_images:
    # mse = np.mean((image_resized - reconstructed_image) ** 2) / (norm_image ** 2)
    mse = np.mean((image_resized - reconstructed_image) ** 2)
    mse_values.append(mse)
mse_values
#plot mse vs compression ratio
plt.plot([int(r*100) for r in compression_ratios], mse_values, marker
='o')
plt.title('MSE vs Comp Ratio using DCT 16x16 blocks')
plt.xlabel('Compression Ratio')
plt.ylabel('Normalized Mean Squared Error')
plt.grid()
plt.show()

```

---

3 Plots and Tables

MSE RESULTS:

Retention Ratio	Block 4×4	Block 8×8	Block 16×16
10%	0.005349	0.003344	0.003003
25%	0.001803	0.001623	0.001483
50%	0.000698	0.000562	0.0005

Figure 1: MSE values for DFT reconstruction with different block sizes and retention ratios

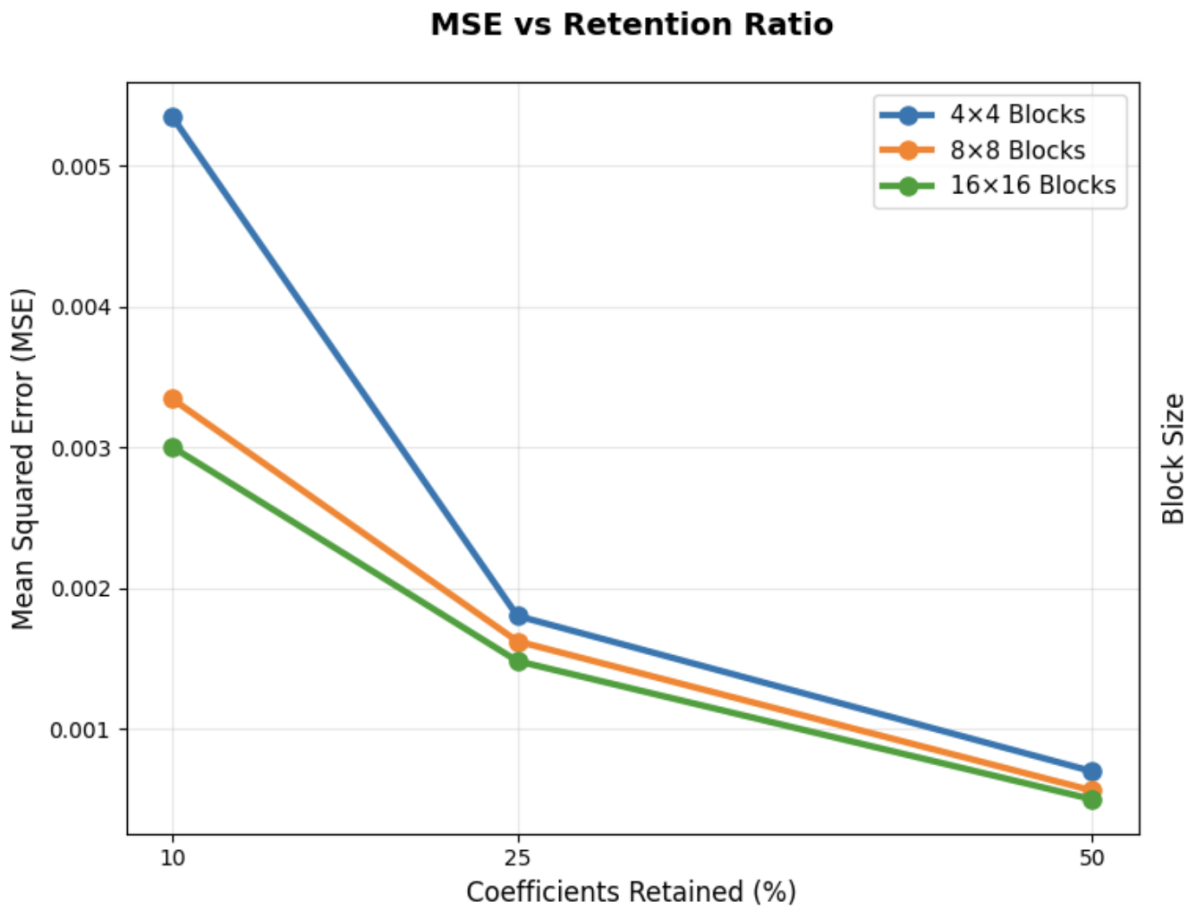


Figure 2: MSE vs. Retention Ratio for different block sizes (DFT)

- MSE decreases as block size increases; larger block sizes mean that each block contains more pixels, capturing more spatial information, so the retained coefficients give a better representation of the image, reducing reconstruction error.
- MSE decreases as the retention ratio increases; higher retention ratios allow for keeping more frequency coefficients, allowing for a better representation of the image.
- So by increasing both decreases the MSE calculated by subtracting the reconstructed from the original, squaring the differences, and averaging over all pixels.

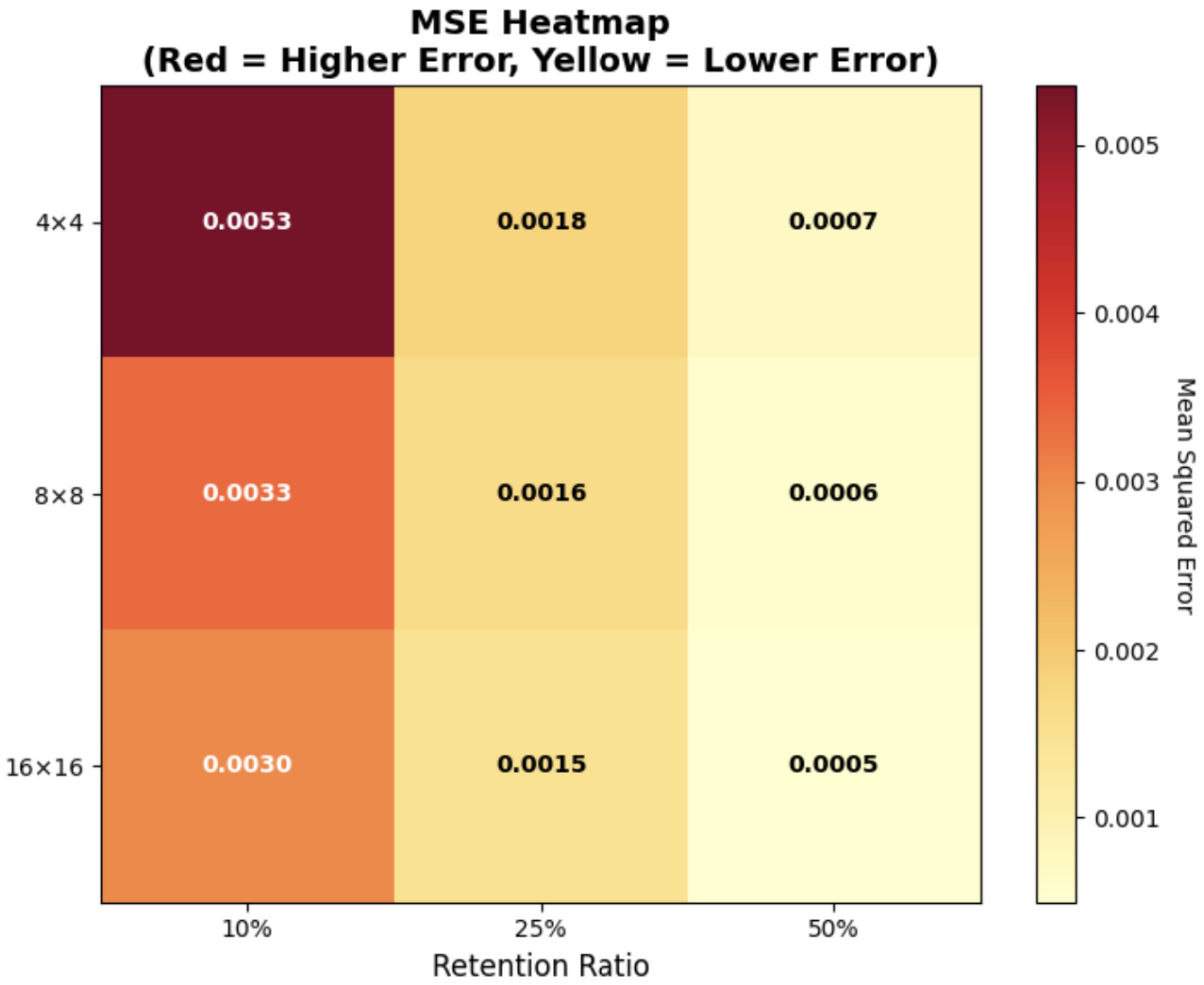


Figure 3: MSE heatmap for different block sizes and retention ratios (DFT).

The heatmap visually shows how MSE changes with block size and retention ratio. Red areas indicate higher reconstruction error, while yellow areas indicate lower error. It helps identify which combinations of block size and retention ratio give acceptable image quality for compression purposes.

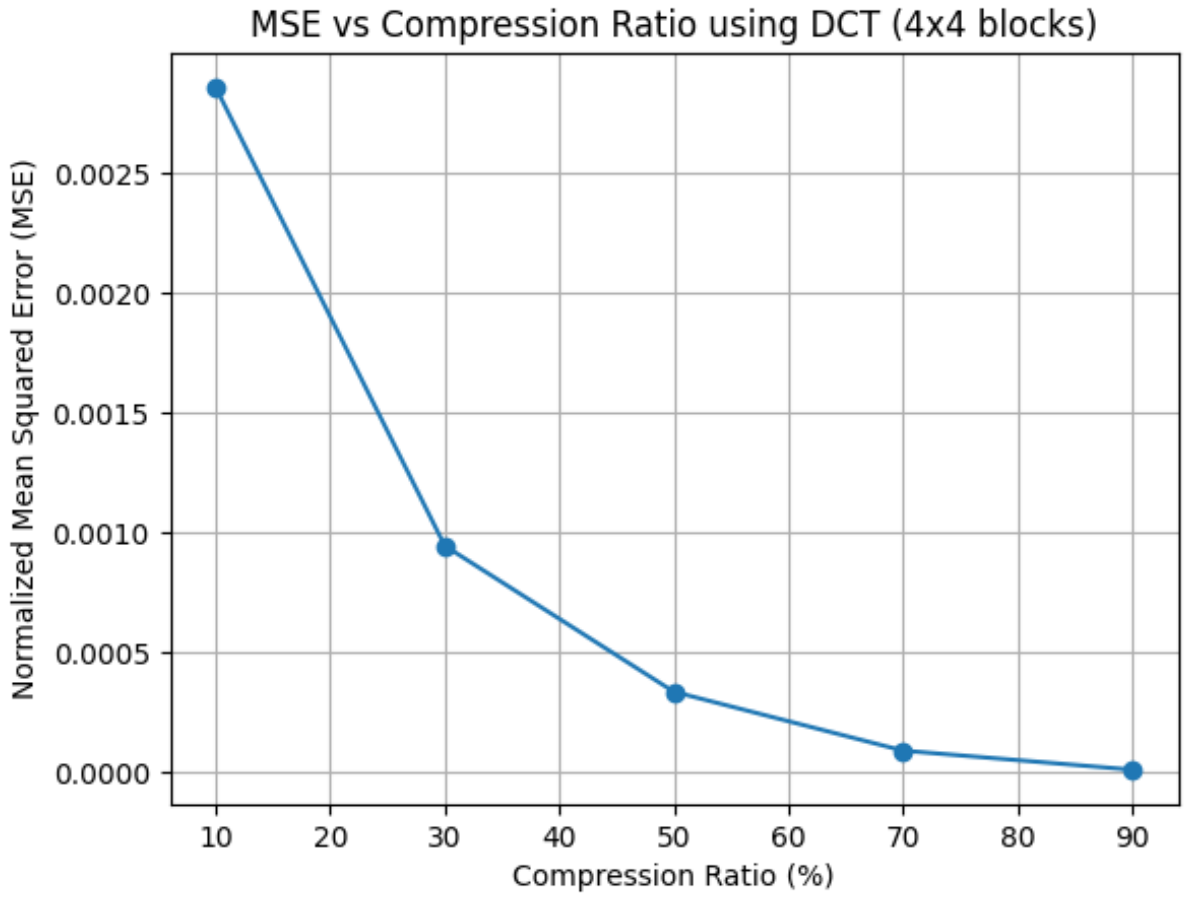


Figure 4: MSE vs. Compression Ratio for 4x4 block size (DCT)

From the plot, as the retention ratio increases for a given block size (in this case 4 x 4), more coefficients are kept, giving a better frequency representation, decreasing the MSE. Hence, the reconstructed image is closer to the original.

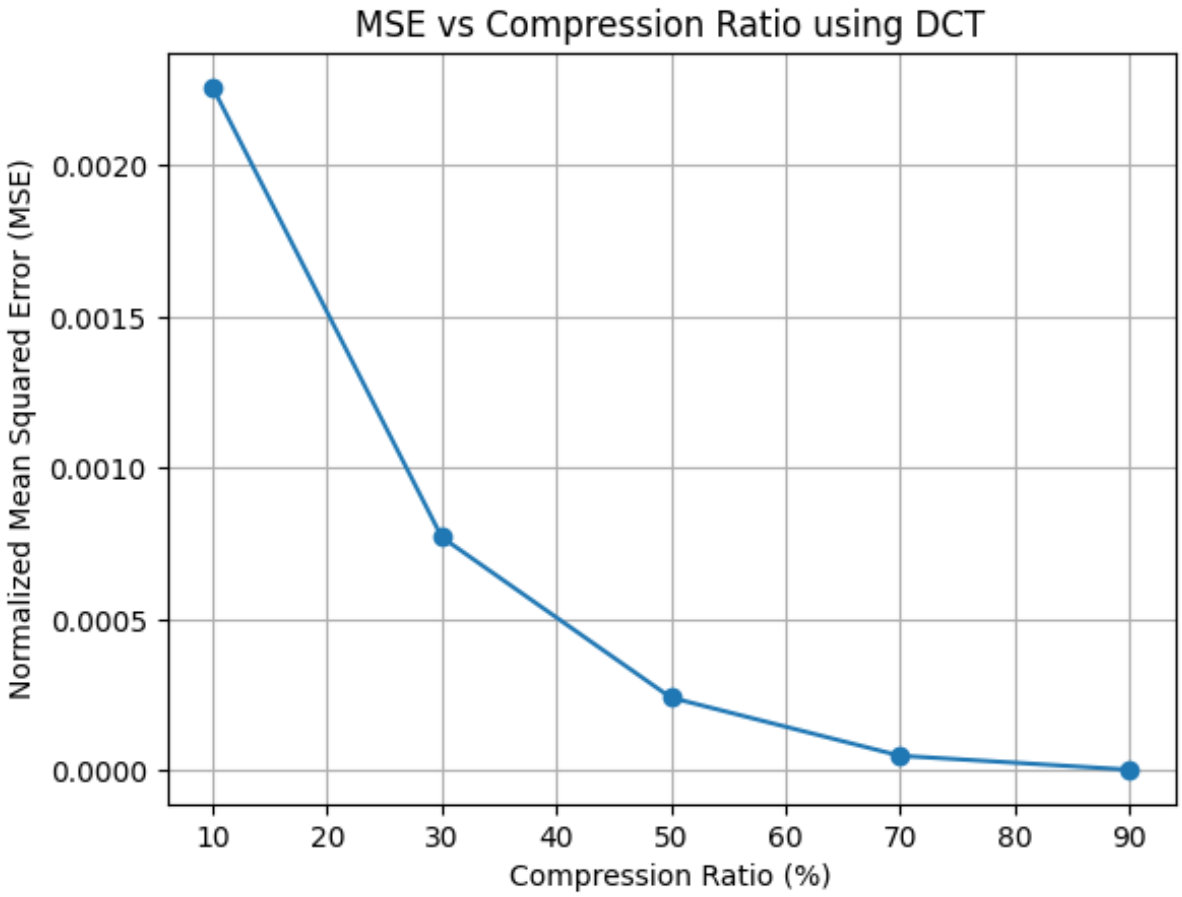


Figure 5: MSE vs. Compression Ratio for 8x8 block size (DCT)

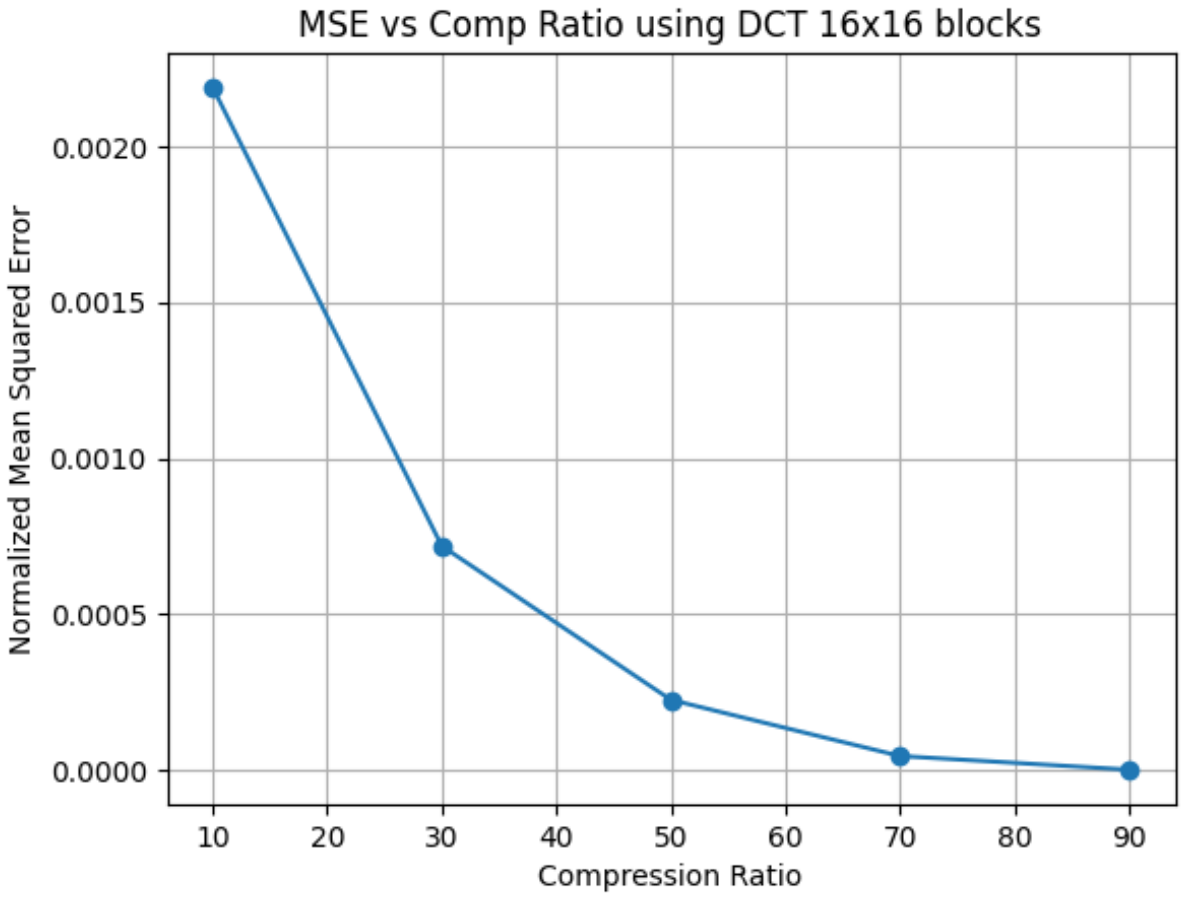


Figure 6: MSE vs. Compression Ratio for 16x16 block size (DCT)

Comparing the 3 plots, we conclude that larger block sizes achieve lower MSE for the same retention ratio because they capture more spatial content due to having more pixels per block. So a single low-frequency coefficient represents the variation over a larger area, capturing more of the image's overall structure, reducing reconstruction errors. When using smaller block sizes, a single low-frequency coefficient represents the variation over a small area, capturing the fine details; where variations appear faster relative to the block size, which means that higher frequency coefficients contain important information for reconstruction, yielding higher MSE at low retention ratios. The choice of block size now presents a tradeoff between capturing local details and overall smoothness.

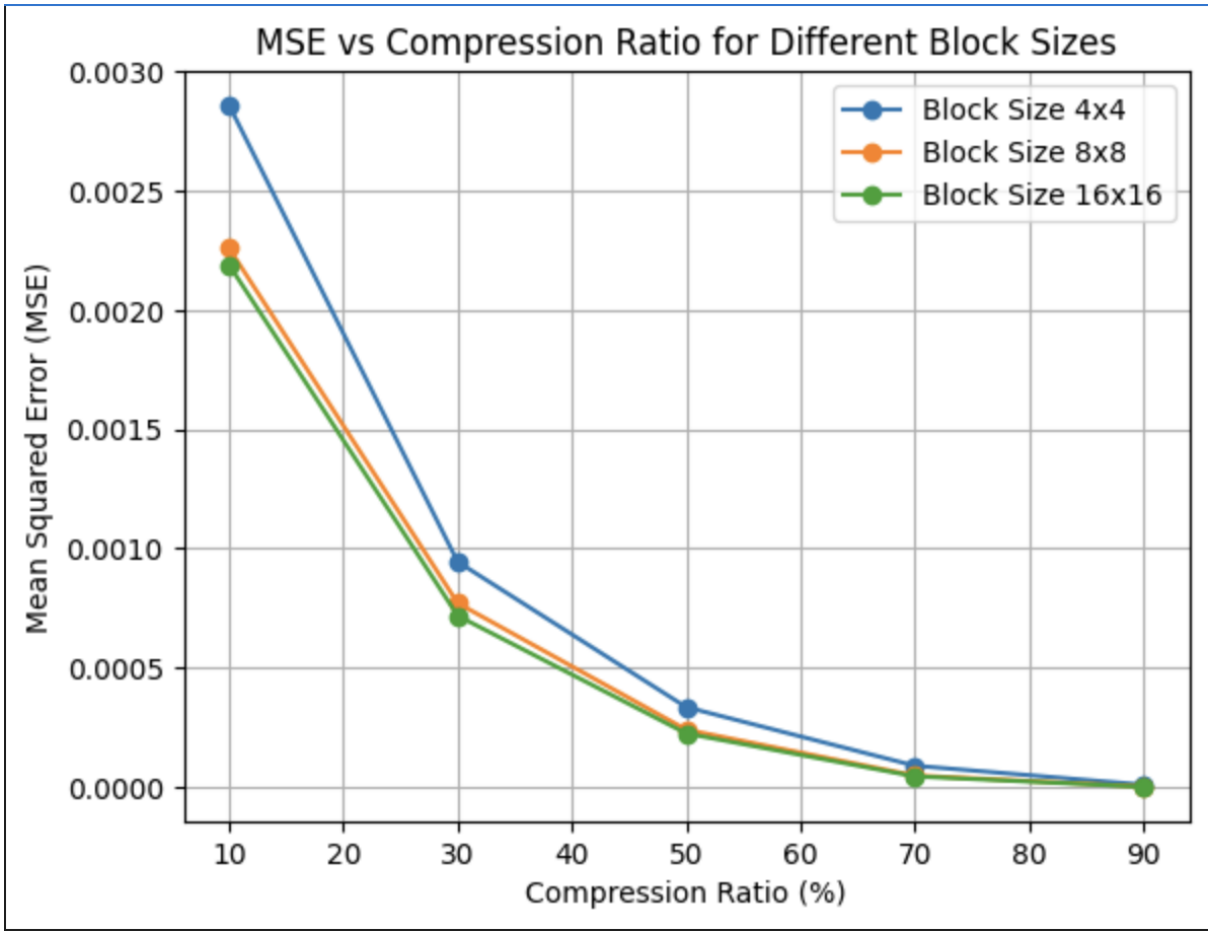


Figure 7: MSE vs. Retention Ratio for different block sizes (DCT)

4 Discussion

4.1 Energy Compaction (Why DCT Performs Better)

Energy compaction is equivalent to how well a transform captures most of the signal's energy in the smallest number of frequency coefficients; pixels in natural images undergo smooth gradual variations where the difference between a pixel and the next isn't significant which is similar to the cosine wave used by the DCT, the cosine wave changes gradually with no sudden oscillations which matches the behavior of pixels in real images. Unlike DFT, which assumes that an image is circular; the last pixel in the block connects to the first pixel which isn't the case, this creates discontinuities at the block boundary which introduces high-frequency components that do not exist in the original image, these aren't easy to remove for compression. DCT fixes this issue by creating an even symmetric extension of the block; it basically mirrors the block. The mirrored half doesn't contain independent information, so we only compute the correlations of the original N samples and the rest follow by symmetry, this symmetry removes the boundary discontinuities and concentrate the signal's energy in a fewer number of coefficients.

4.2 Effect of Block Size

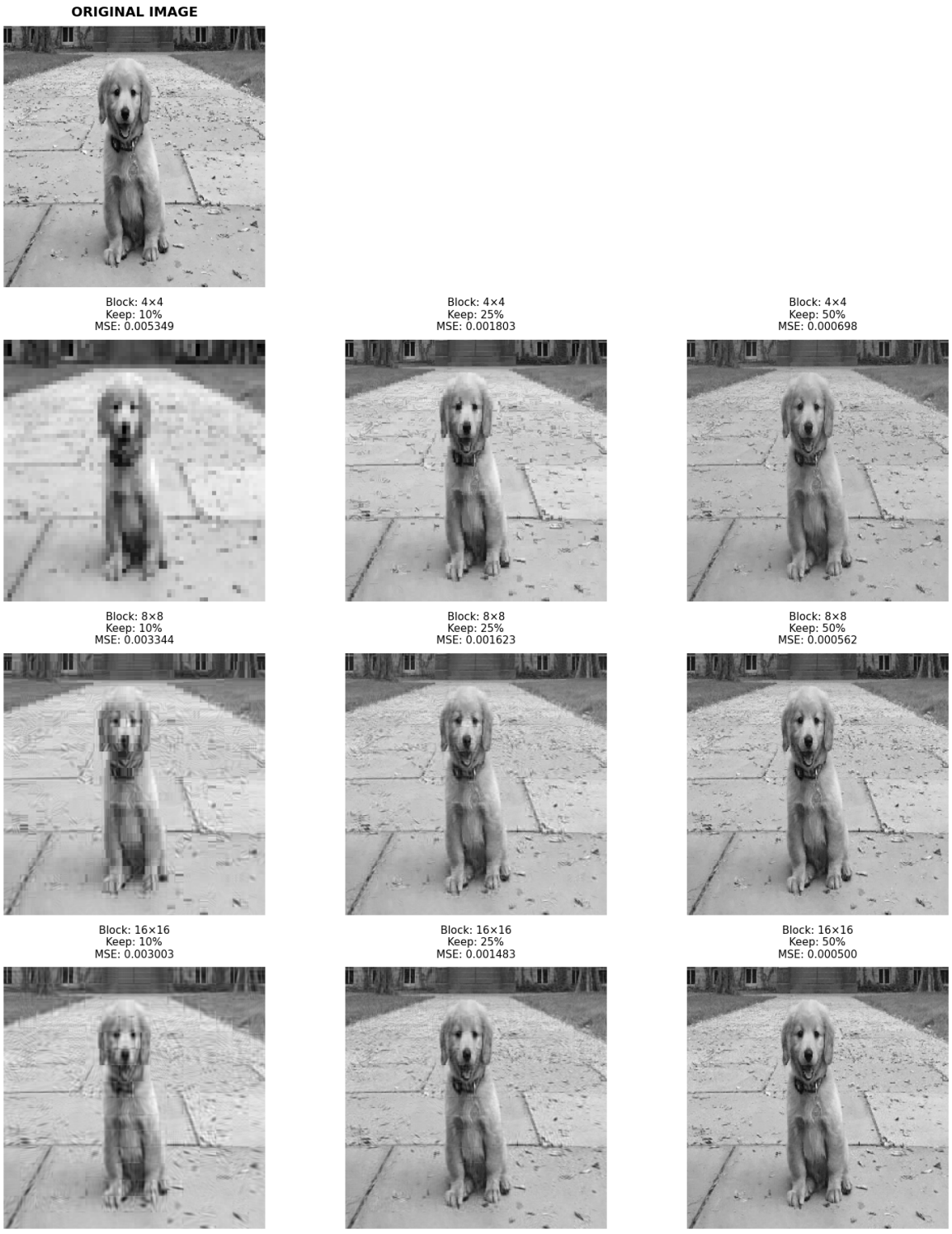


Figure 8: Comparison grid of DFT results



(a) 4×4 DCT (b) 8×8 DCT (c) 16×16 DCT

Figure 9: Comparison of DCT reconstructions with different block sizes



(a) 4×4 DCT (b) 8×8 DCT (c) 16×16 DCT

Figure 10: Comparison of DCT reconstructions with different block sizes



(a) 4×4 DCT (b) 8×8 DCT (c) 16×16 DCT

Figure 11: Comparison of DCT reconstructions with different block sizes

Looking at the vertical columns for DFT, or the separate images for DCT:

Block size directly impacts the quality of reconstruction. When an image is divided using any arbitrary block size into blocks, the transform (whether DFT or DCT) is applied independently to each block. Thus, the block size determines the amount of spatial information the transform captures.

4×4 Blocks

A single low-frequency coefficient represents the variation over a small area, capturing the fine details, where variations appear faster relative to the block size. This means that

higher-frequency coefficients contain important information for reconstruction, yielding higher MSE at low retention ratios.

8×8 Blocks

Creates a balance between fine detail and overall smoothness. Low-frequency coefficients represent more of the signal's energy, thus carrying more impactful information for reconstruction. This results in lower MSE for the same retention ratio compared to the 4×4 case.

16×16 Blocks

Each block contains more pixels, allowing the transform to capture much more spatial information. Low-frequency coefficients represent variations across a large region, capturing most of the signal's energy and giving a reconstruction similar to the original. This results in noticeably lower MSE for the same retention ratio compared to the above sizes.

4.3 Visual Effect of Retaining Too Few Coefficients

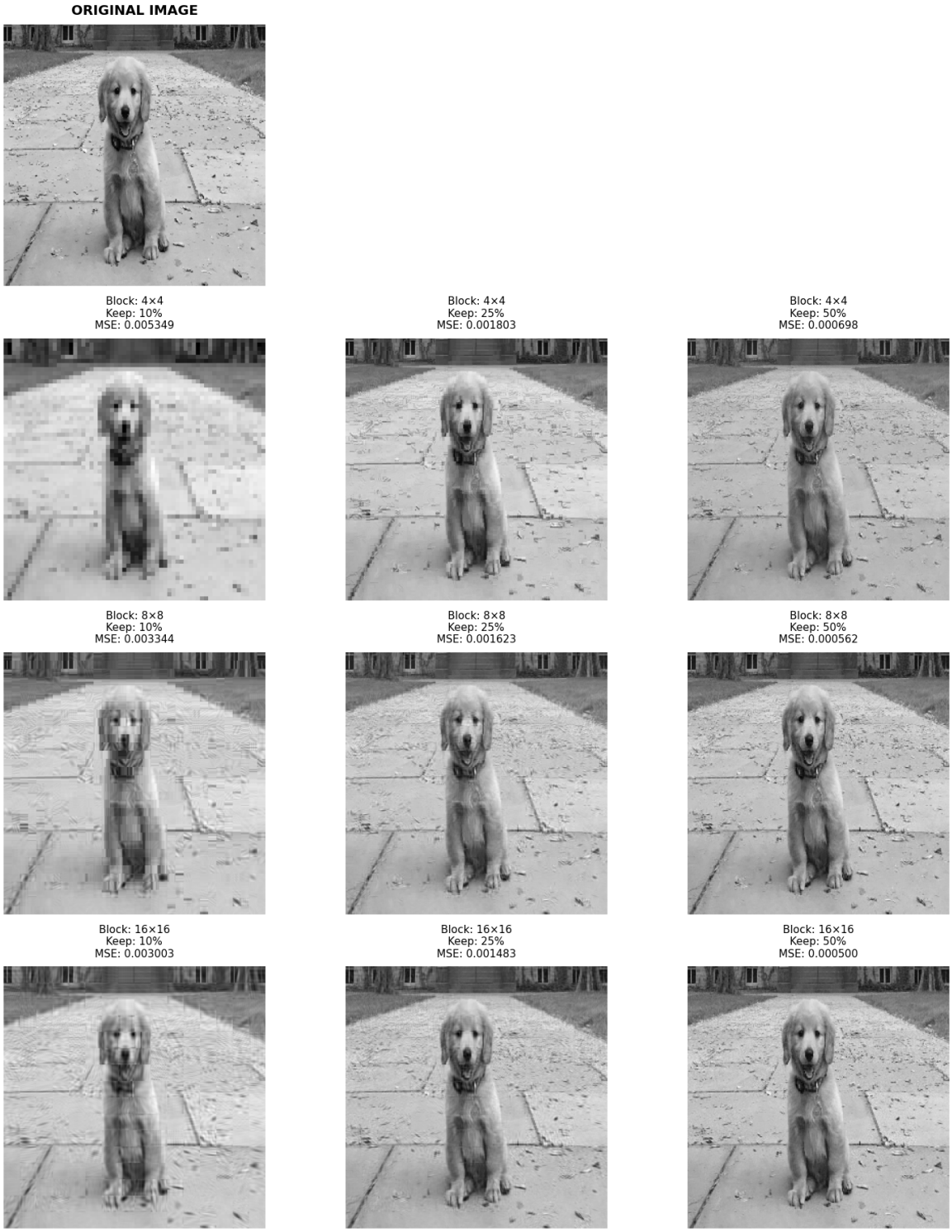


Figure 12: Comparison grid of DFT results



(a) 4×4 DCT (b) 8×8 DCT (c) 16×16 DCT

Figure 13: Comparison of DCT reconstructions with different block sizes



(a) 4×4 DCT (b) 8×8 DCT (c) 16×16 DCT

Figure 14: Comparison of DCT reconstructions with different block sizes

Looking at the horizontal rows for DFT, or separate images for DCT:

When too few frequency components are retained in either DFT or DCT reconstruction, the general shape of the image is preserved, but loss of details occurs, basically the image gets blurry as the retention ratio decreases. When larger block sizes are used, this could make the edges of each block less smooth relative to its neighbors; adjacent blocks are processed independently so the frequency approximation in one block might not match the frequency approximation of the other due to retaining too few coefficients, which could have smoothed barriers across block boundaries. Because of this mismatch, there could occur sudden changes in intensity, creating a blocky look. In conclusion, keeping only a few frequency coefficients preserves the general shape of the image, but causes a lack of sharpness and clarity.

5 Bonus

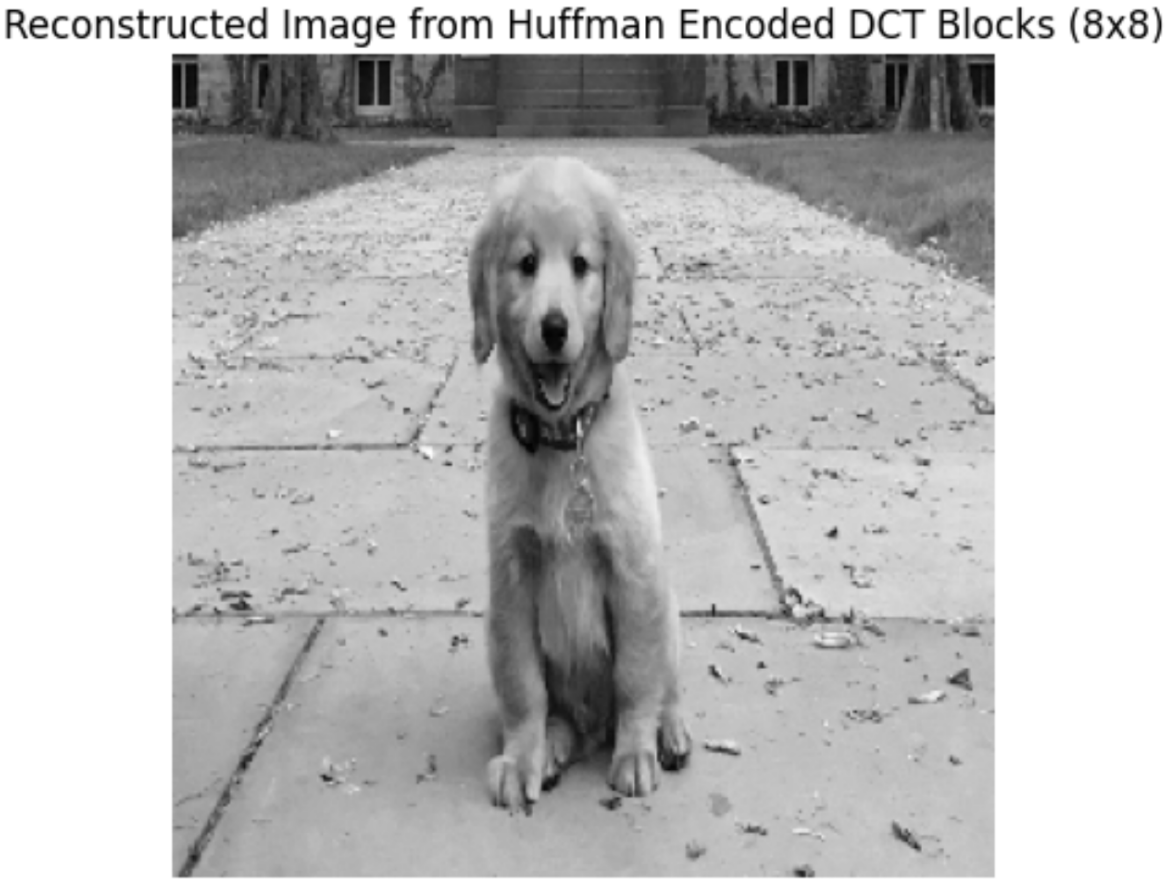


Figure 15: Reconstructed image from Huffman encoded DCT blocks(8x8)

The MSE of the reconstructed image from Huffman encoded DCT blocks is 5.65×10^{-32}.